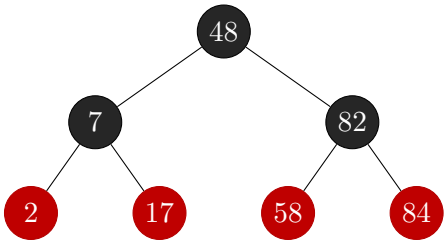
Benchmark Comparatif

Arbre rouge-noir

ABR, rouge-noir, `std::map` et `std::unordered_map`

Insertion, recherche et suppression de clés entières :
cas normal et pire cas de chaque structure



Auteur : BENNOUR RAFAA
Formation : ILISI – 2e année
Date : 13 novembre 2026
Langage : C++17
Compilateur : g++ 15.2 `-O2`

Table des matières

1 Introduction **3**
- 1.1 Contexte et objectifs . . . 3
- 1.2 L'arbre rouge-noir . . . 4
- 1.3 La table de hachage et ses collisions . . . 5
- 1.4 Jeux de données . . . 5
- 1.5 Opérations mesurées . . . 6
- 1.6 Complexités théoriques . . . 6

2 Méthodologie **7**
- 2.1 Environnement . . . 7
- 2.2 Protocole de mesure . . . 7
- 2.3 Implémentation . . . 7

3 Résultats : démonstration **11**
- 3.1 Insertion . . . 11
- 3.2 Suppression . . . 11

4 Résultats : clés en ordre aléatoire **13**
- 4.1 Données brutes . . . 13
- 4.2 Visualisation . . . 14
- 4.3 Analyse . . . 14

5 Résultats : clés en ordre croissant **16**
- 5.1 Données brutes . . . 16
- 5.2 Visualisation . . . 17
- 5.3 Analyse . . . 17

6 Résultats : collisions dans la table de hachage **18**
- 6.1 Données brutes . . . 18
- 6.2 Visualisation . . . 19
- 6.3 Analyse . . . 20

7 Analyse comparative approfondie **21**
- 7.1 Notre arbre rouge-noir face à `std::map` . . . 21
- 7.2 ABR ou rouge-noir : le prix de l'équilibrage . . . 21
- 7.3 Les arbres face à la table de hachage . . . 21
- 7.4 Le pire cas de chaque structure . . . 22
- 7.5 Mémoire . . . 22
- 7.6 Bilan . . . 23

8 Conclusion **24**
8.1 Synthèse des résultats 24
8.2 Recommandations 24
8.3 Conclusion finale 25

Chapitre 1

Introduction

1.1 Contexte et objectifs

Un arbre binaire de recherche (ABR) range les valeurs plus petites à gauche et les plus grandes à droite. Chercher, insérer ou supprimer une valeur coûte autant d'étapes que la **hauteur** de l'arbre. Le problème : si les valeurs arrivent triées, l'ABR devient une liste et sa hauteur vaut n.

L'**arbre rouge-noir** corrige ce défaut : il donne une couleur à chaque nœud et se répare après chaque insertion et chaque suppression, par des recolorations et des rotations. Sa hauteur reste toujours inférieure à $2\log_2(n+1)$.

Ce rapport compare quatre structures qui permettent de ranger des clés, de les retrouver et de les supprimer :

ABR
arbre binaire de recherche simple, **non équilibré**, écrit à la main ;

Rouge-noir
arbre rouge-noir **écrit à la main** (insertion, suppression, rotations) ;

`std::map`
l'arbre rouge-noir de la bibliothèque standard ;

`std::unordered_map`
la table de hachage de la bibliothèque standard.

Chaque structure est testée dans le cas normal (clés mélangées) et dans son **pire cas** : des clés triées pour l'ABR, des clés qui entrent en collision pour la table de hachage.

1.1.1 Objectifs de l'étude

1. Implémenter un arbre rouge-noir complet (insertion, suppression, rotations, recoloration) et vérifier qu'il respecte ses règles.
2. Mesurer l'effet de l'équilibrage, en insérant les clés dans un ordre **aléatoire** puis **croissant**.
3. Comparer notre arbre rouge-noir avec celui de la bibliothèque standard (`std::map`).
4. Situer les arbres face à une table de hachage (`std::unordered_map`), sans collisions puis avec des **collisions** de plus en plus nombreuses.

1.2 L'arbre rouge-noir

1.2.1 Les cinq règles

1. Chaque nœud est rouge ou noir.
2. La racine est noire.
3. Les feuilles vides (NIL, `nullptr`) sont considérées comme noires.
4. Un nœud rouge n'a jamais d'enfant rouge.
5. Depuis un nœud, tous les chemins vers les feuilles passent par le même nombre de nœuds noirs.

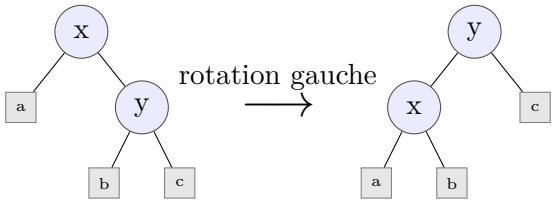
Les règles 4 et 5 garantissent l'équilibre : tous les chemins ont le même nombre b de nœuds noirs, et on ne peut pas placer deux rouges à la suite, donc le chemin le plus long fait au plus $2b$ nœuds : il est **au plus deux fois plus long que le plus court**.

1.2.2 L'insertion

On insère comme dans un ABR, avec un nœud **rouge**. Si son parent est rouge, la règle 4 est cassée et on répare selon la couleur de l'**oncle** (le frère du parent) :

Cas	Situation	Réparation
Cas 1	oncle rouge	parent et oncle deviennent noirs, grand-parent rouge ; on recommence avec le grand-parent
Cas 2	oncle noir, nœud en zigzag	rotation sur le parent pour se ramener au cas 3
Cas 3	oncle noir, nœud en ligne droite	rotation sur le grand-parent, puis échange des couleurs ; terminé

Une **rotation** ne modifie que trois liens, elle coûte $O(1)$:



Une insertion fait au plus deux rotations et $O(\log n)$ recolorations : elle coûte $O(\log n)$.

1.2.3 La suppression

On retire d'abord le nœud comme dans un ABR : s'il a deux enfants, son **successeur** (le minimum du sous-arbre droit) prend sa place et sa couleur. Si le nœud qui quitte vraiment sa place est **rouge**, aucune règle n'est cassée. S'il est **noir**, il manque un noir sur certains chemins (règle 5) : le nœud x qui prend la place reçoit un « noir en trop » (on dit qu'il est **doublement noir**). Si x est rouge, il devient simplement noir. Sinon, on répare selon le **frère** de x et ses enfants (les **neveux**) :

Cas	Situation	Réparation
Cas 1	frère rouge	rotation sur le parent et échange des couleurs : le frère devient noir, on passe aux cas 2 à 4
Cas 2	frère noir, deux neveux noirs	le frère devient rouge ; le noir en trop remonte au parent
Cas 3	frère noir, neveu proche rouge, neveu lointain noir	rotation sur le frère pour se ramener au cas 4
Cas 4	frère noir, neveu lointain rouge	rotation sur le parent et recoloration ; terminé

Une suppression fait au plus trois rotations et $O(\log n)$ recolorations : elle coûte aussi $O(\log n)$, mais elle fait plus de travail qu'une insertion (chercher le successeur, puis réparer).

1.3 La table de hachage et ses collisions

Une table de hachage est un tableau de B **cases**. Pour ranger une clé, on calcule son numéro de case avec une fonction de hachage ; avec g++, pour un entier, cette fonction renvoie la clé elle-même, et la case vaut simplement clé mod B. Si deux clés tombent dans la même case, on parle de **collision** : les clés de la case sont alors chaînées dans une liste, qu'il faut parcourir élément par élément.

Quand les clés sont bien réparties, chaque case contient environ une clé et chaque opération coûte $O(1)$. Mais si k clés s'entassent dans la même case, chaque opération sur cette case coûte $O(k)$; dans le pire cas, toutes les clés sont dans la même case et la table n'est plus qu'une liste : $O(n)$.

1.4 Jeux de données

— $N = 10^3$, 10^4, 10^5, 10^6 et 2×10^6 clés entières, sans doublon.

— **Ordre aléatoire** : les clés $0, 1, \ldots, N-1$ sont mélangées (graine fixe). C'est le cas normal.

— **Ordre croissant** : les clés $0, 1, \ldots, N-1$ sont insérées triées. C'est le pire cas de l'ABR, qui n'est mesuré que jusqu'à $N = 10\,000$ (au-delà, son coût en $O(N^2)$ devient trop long).

— **Collisions** : on fixe le nombre de cases B de la table dès sa création (`reserve(N)`), pour qu'elle ne se redimensionne jamais, puis on choisit des clés qui mettent exactement k clés dans chaque case utilisée : dans la case c, on place c, $c + B$, $c + 2B$, $\ldots$, $c + (k-1)B$, qui ont toutes le même reste modulo B. On teste $k = 1$ (aucune collision), 10, 100 et 1000. Les clés sont ensuite mélangées. Les arbres reçoivent les mêmes clés : comme ils ne regardent que l'*ordre* des clés, et non leur case, ils ne sont pas gênés.

— Démonstration : insertion de 7 58 48 2 17 84 82, puis suppression de 2, 58, 48, 7 et 99 (absent).

1.5 Opérations mesurées

— **Insertion** : insérer les N clés dans une structure vide.

— **Recherche** : chercher les N clés, dans un autre ordre aléatoire. Toutes doivent être trouvées.

— **Suppression** : supprimer les N clés, dans ce même ordre aléatoire, jusqu'à vider la structure. Toutes doivent être supprimées.

— **Hauteur** : nombre de niveaux des deux arbres écrits à la main, mesuré avant les suppressions (elle n'est pas accessible pour `std::map`).

Chaque résultat est un temps moyen par opération, en nanosecondes (ns).

1.6 Complexités théoriques

TABLE 1.1 – Complexités (cas moyen, avec le pire cas entre parenthèses).

Structure	Insertion, recherche, suppression	Ordre des clés	Pire cas provoqué par
ABR	$O(\log n)$ $(O(n))$	trié	clés triées
Rouge-noir / `map`	$O(\log n)$ garanti	trié	aucun
`unordered_map`	$O(1)$ $(O(n))$	aucun	collisions

Chapitre 2

Méthodologie

2.1 Environnement

— Ordinateur portable : Intel Core i7-8550U, 8 Go de mémoire, Windows 11.

— Compilateur MinGW g++ 15.2, options `-O2 -std=c++17`.

— Les temps dépendent de la machine ; les tendances restent les mêmes d'un PC à l'autre.

2.2 Protocole de mesure

— Chronomètre : `std::chrono::steady_clock`.

— Chaque mesure est faite 3 fois et l'on garde le meilleur temps.

— Les quatre structures reçoivent exactement les mêmes clés et les mêmes requêtes.

— **Vérifications** avant les mesures, sur 100 000 clés : l'arbre rouge-noir respecte ses règles après les insertions (aléatoires et croissantes), puis après la suppression d'une clé sur deux ; les bonnes clés sont encore présentes ; et l'arbre est vide une fois toutes les clés supprimées.

— **Vérifications** pendant les mesures : chaque clé cherchée est trouvée et chaque clé supprimée était bien présente. Dans le scénario collisions, le programme vérifie aussi que la table n'a pas changé de taille et que la case la plus remplie contient exactement k clés.

— Durée totale du benchmark : environ 20 minutes, dont la plus grande partie pour les collisions à $k = 1000$ avec 10^6 et 2×10^6 clés. L'option `rapide` (`Benchmark rapide`) ne mesure que $N = 1\,000$ et permet de vérifier le programme en quelques secondes.

— **Affichage** : pour chaque scénario, le programme imprime un tableau par opération (insertion, recherche, suppression), avec les structures en colonnes et les tailles N en lignes, puis un tableau des hauteurs. La sortie complète est dans `resultats/benchmark.txt`.

2.3 Implémentation

2.3.1 Le nœud

Le nœud rouge-noir a un pointeur vers son **parent**, nécessaire pour remonter vers le grand-parent, l'oncle ou le frère.

```
enum Couleur { ROUGE, NOIR };
struct NoeudRN {
    int valeur;
    Couleur couleur = ROUGE;               // tout nouveau noeud est rouge
    NoeudRN *gauche = nullptr, *droite = nullptr, *parent = nullptr;
};
static Couleur couleurDe(NoeudRN* n) { return n == nullptr ? NOIR : n->couleur; }
```

2.3.2 La rotation gauche

```
void rotationGauche(NoeudRN* x) {
    NoeudRN* y = x->droite;
    x->droite = y->gauche;                          // 1. b passe à droite de x
    if (y->gauche != nullptr) y->gauche->parent = x;
    y->parent = x->parent;                          // 2. y prend la place de x
    if (x->parent == nullptr)        racine = y;
    else if (x == x->parent->gauche) x->parent->gauche = y;
    else                             x->parent->droite = y;
    y->gauche = x;                                  // 3. x passe à gauche de y
    x->parent = y;
}
```

2.3.3 La réparation après insertion

Le code ci-dessous traite le cas où le parent est l'enfant gauche du grand-parent ; l'autre côté est son miroir (gauche et droite inversés).

```
while (z != racine && couleurDe(z->parent) == ROUGE) {
    NoeudRN* p = z->parent;  NoeudRN* g = p->parent;
    if (p == g->gauche) {
        NoeudRN* oncle = g->droite;
        if (couleurDe(oncle) == ROUGE) {           // CAS 1 : recolorer
            p->couleur = NOIR; oncle->couleur = NOIR; g->couleur = ROUGE;
            z = g;                                 // on remonte
        } else {
            if (z == p->droite) {                  // CAS 2 : zigzag
                z = p; rotationGauche(z); p = z->parent;
            }
            p->couleur = NOIR; g->couleur = ROUGE; // CAS 3 : ligne droite
            rotationDroite(g);
        }
    } else { /* miroir */ }
}
racine->couleur = NOIR;
```

2.3.4 La suppression

Les nœuds sont déplacés en changeant les pointeurs (`transplanter`), jamais en recopiant les valeurs : seul le nœud supprimé disparaît.

```
Couleur couleurRetiree = z->couleur;
if (z->gauche == nullptr) {                  // un seul enfant (ou aucun)
    x = z->droite; parentX = z->parent; transplanter(z, z->droite);
} else if (z->droite == nullptr) {
    x = z->gauche; parentX = z->parent; transplanter(z, z->gauche);
} else {                                     // deux enfants : y remplace z
    NoeudRN* y = minimum(z->droite);
    couleurRetiree = y->couleur;  x = y->droite;
    if (y->parent == z) parentX = y;
    else { parentX = y->parent; transplanter(y, y->droite);
           y->droite = z->droite; y->droite->parent = y; }
    transplanter(z, y);
    y->gauche = z->gauche; y->gauche->parent = y;
    y->couleur = z->couleur;
}
delete z;
if (couleurRetiree == NOIR) reparerSuppression(x, parentX);  // il manque un noir
```

2.3.5 La réparation après suppression

Comme pour l'insertion, seul le côté gauche est montré ; l'autre côté est son miroir. Le nœud x peut être vide, c'est pourquoi on garde aussi son parent p.

```
while (x != racine && couleurDe(x) == NOIR) {
    if (x == p->gauche) {
        NoeudRN* frere = p->droite;
        if (couleurDe(frere) == ROUGE) {                    // CAS 1
            frere->couleur = NOIR; p->couleur = ROUGE;
            rotationGauche(p); frere = p->droite;
        }
        if (couleurDe(frere->gauche) == NOIR && couleurDe(frere->droite) == NOIR) {
            frere->couleur = ROUGE; x = p; p = x->parent;     // CAS 2 : on remonte
        } else {
            if (couleurDe(frere->droite) == NOIR) {           // CAS 3
                frere->gauche->couleur = NOIR; frere->couleur = ROUGE;
                rotationDroite(frere); frere = p->droite;
            }
            frere->couleur = p->couleur; p->couleur = NOIR;   // CAS 4 : terminé
            frere->droite->couleur = NOIR;
            rotationGauche(p); x = racine;
        }
    } else { /* miroir */ }
}
if (x != nullptr) x->couleur = NOIR;
```

2.3.6 Les collisions dans la table de hachage

Pour que les collisions voulues restent des collisions, la table ne doit jamais se redimensionner pendant le test. On fixe donc son nombre de cases dès la construction, puis on fabrique les clés à partir de ce nombre.

```cpp
int CASES_RESERVEES = 0;
struct TableReservee : unordered_map<int, int> {
    TableReservee() { reserve(CASES_RESERVEES); }   // B cases, fixées
};

// k clés par case : la case c reçoit c, c+B, c+2B, ...
vector<int> clesAvecCollisions(int N, int k, long long B) {
    int M = (N + k - 1) / k;            // cases utilisées
    vector<int> cles(N);
    for (int i = 0; i < N; i++)
        cles[i] = (int)(i % M + (long long)(i / M) * B);
    shuffle(cles.begin(), cles.end(), mt19937(42));
    return cles;
}
```

Par exemple, pour $N = 1\,000$, la table a $B = 1\,031$ cases. Avec $k = 10$, on utilise 100 cases et on met 10 clés dans chacune ; avec $k = 1000$, les mille clés $0, 1031, 2062, \ldots$ sont toutes dans la case 0.

2.3.7 Organisation du projet

Fichier	Rôle
`ArbreBinaireRecherche.h`	ABR non équilibré
`ArbreRougeNoir.h`	arbre rouge-noir et vérification des règles
`DemoRBT.cpp`	insertion puis suppression pas à pas
`Benchmark.cpp`	comparaison des quatre structures dans les trois scénarios
`resultats/benchmark.txt`	sortie complète du benchmark
`resultats/demo_rbt.txt`	sortie de la démonstration
`rapport/rapport_rbt.tex`	source de ce rapport

Chapitre 3

Résultats : démonstration

3.1 Insertion

Pour la suite 7 58 48 2 17 84 82, le programme affiche l'arbre ci-dessous (racine à gauche, R = rouge, N = noir). C'est le même arbre que celui obtenu à la main.

```
            84R
      82N
            58R
48N
            17R
      7N
            2R
Regles respectees : oui
```

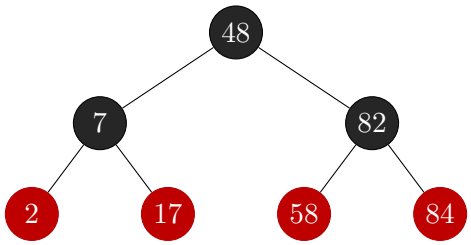


FIGURE 3.1 – Arbre après les insertions : chaque chemin compte deux nœuds noirs.

Les étapes : 48 provoque un zigzag (cas 2 puis cas 3), 2 une recoloration (cas 1), 17 et 84 ne demandent rien, et 82 un nouveau zigzag.

3.2 Suppression

Suppression	Ce qui se passe
2 et 58	feuilles rouges : on les retire, aucune règle n'est cassée
48 (racine)	deux enfants : le successeur 82 prend sa place et sa couleur noire ; 82 était noir, donc il manque un noir sous lui, mais 84, qui monte à sa place, est rouge : il devient noir
7	un seul enfant, 17 (rouge), qui monte et devient noir
99	absente : la fonction renvoie `false`, l'arbre ne change pas

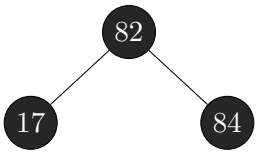


FIGURE 3.2 – Arbre final après les suppressions : les règles sont toujours respectées.

Cette démonstration ne passe que par les cas simples. Les quatre cas de réparation (frère rouge, neveux noirs, etc.) sont exercés par la vérification du benchmark, qui supprime 50 000 clés dans un ordre aléatoire et contrôle ensuite les règles et le contenu de l'arbre.

Chapitre 4

Résultats : clés en ordre aléatoire

4.1 Données brutes

TABLE 4.1 – Ordre aléatoire : temps moyen par opération (ns). En vert, l'arbre le plus rapide.

N	Structure	Hauteur	Insertion	Recherche	Suppression
1 000	ABR	22	133.0	29.7	88.1
	Rouge-noir	12	123.2	19.5	84.2
	`map`	–	133.5	22.7	112.3
	`unordered_map`	–	129.4	7.4	49.0
10 000	ABR	29	156.8	58.4	127.0
	Rouge-noir	16	218.9	42.3	141.3
	`map`	–	164.5	49.6	173.1
	`unordered_map`	–	104.6	8.7	62.2
100 000	ABR	41	281.2	135.9	284.5
	Rouge-noir	20	350.8	161.2	384.7
	`map`	–	448.7	165.2	448.0
	`unordered_map`	–	173.2	15.6	143.4
1 000 000	ABR	50	682.8	526.8	681.6
	Rouge-noir	24	602.6	362.2	771.0
	`map`	–	670.8	404.3	896.8
	`unordered_map`	–	269.5	60.1	342.5
2 000 000	ABR	57	765.3	669.2	789.4
	Rouge-noir	25	741.9	401.6	928.4
	`map`	–	800.6	477.8	1 033.4
	`unordered_map`	–	311.3	68.4	415.2

4.2 Visualisation

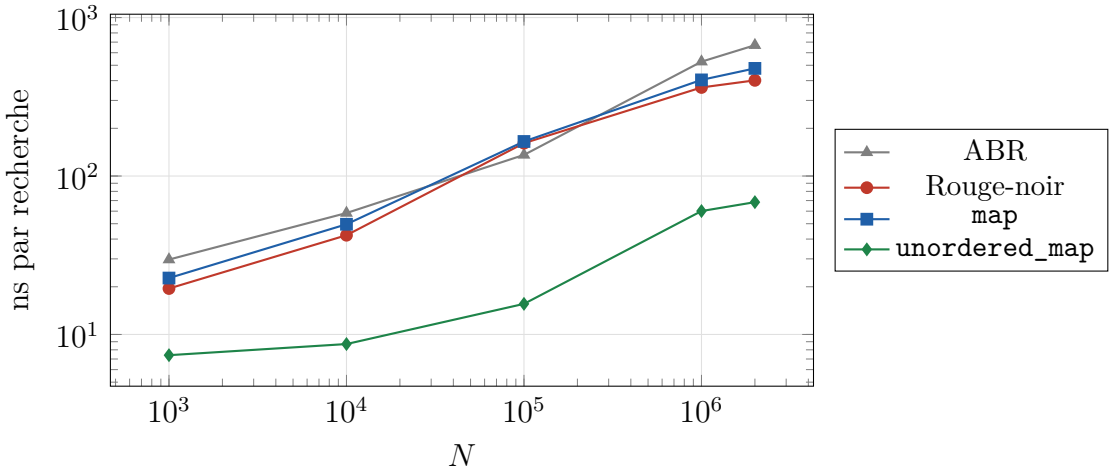


FIGURE 4.1 – Recherche, clés en ordre aléatoire (échelles logarithmiques).

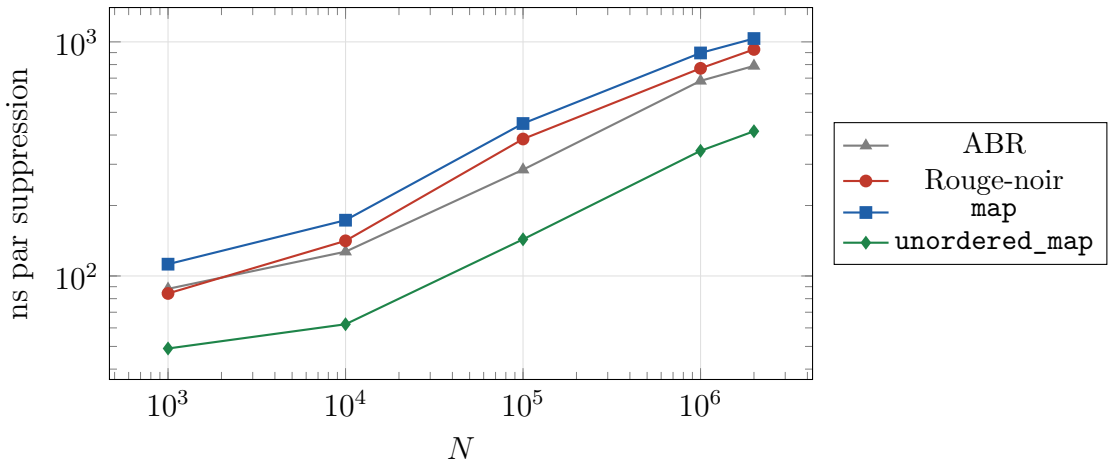


FIGURE 4.2 – Suppression, clés en ordre aléatoire (échelles logarithmiques).

4.3 Analyse

— **Hauteur** : avec des clés mélangées, l'ABR reste assez équilibré (57 niveaux pour deux millions de clés), mais le rouge-noir est **deux fois moins haut** (25 niveaux, proche de $\log_2(2 \times 10^6) \approx 21$).

— **Recherche** : le rouge-noir est le plus rapide des arbres pour les grandes tailles. À 2×10^6, il cherche **1,7 fois plus vite** que l'ABR (402 contre 669 ns) : la recherche ne fait aucune réparation et profite entièrement de la hauteur plus faible.

— **Insertion** : les trois arbres sont proches, et le plus rapide change selon la taille (l'ABR à 10^4 et 10^5, notre rouge-noir à 10^3, 10^6 et 2×10^6). L'écart ne dépasse jamais 25 %.

— **Suppression** : l'ABR est le plus rapide des arbres à partir de 10^4 (789 contre 928 ns pour le rouge-noir à 2×10^6, soit 15 % de moins). Avec des clés mélangées, il reste assez bas, et il n'a **aucune réparation** à faire, alors que le rouge-noir doit recolorer et parfois tourner après avoir retiré un nœud noir.

— Pour le rouge-noir et `map`, la suppression est l'opération la plus chère : il faut chercher le nœud, puis son successeur, puis réparer. Pour l'ABR, qui ne répare rien, elle coûte à peu près autant que l'insertion.

— Entre 10^6 et 2×10^6, les temps augmentent peu : la hauteur ne gagne qu'un niveau, et le coût est surtout dû aux accès mémoire (défauts de cache), déjà présents à 10^6.

— Les mesures sont un peu bruitées : à 10^5, l'ABR est devant partout, y compris en recherche (136 contre 161 ns) alors qu'il est deux fois plus haut, ce qui n'a pas de raison théorique. Les écarts de moins de 20 % ne sont pas significatifs.

Chapitre 5

Résultats : clés en ordre croissant

5.1 Données brutes

Table 5.1 – Ordre croissant : temps moyen par opération (ns). « – » : non mesuré (trop long) ou sans objet.

N	Structure	Hauteur	Insertion	Recherche	Suppression
1 000	ABR	1 000	1 091.4	1 219.0	546.3
	Rouge-noir	17	128.5	23.9	83.9
	`map`	–	128.4	26.5	112.0
	`unordered_map`	–	121.2	7.3	43.7
10 000	ABR	10 000	12 377.3	15 724.3	7 067.9
	Rouge-noir	24	90.5	41.2	115.0
	`map`	–	89.5	48.5	139.3
	`unordered_map`	–	72.7	7.9	55.6
100 000	ABR	–	–	–	–
	Rouge-noir	31	136.4	96.1	206.8
	`map`	–	129.3	93.3	219.0
	`unordered_map`	–	94.1	13.1	74.5
1 000 000	ABR	–	–	–	–
	Rouge-noir	37	154.1	289.9	593.3
	`map`	–	152.3	342.0	671.2
	`unordered_map`	–	101.6	61.1	321.4
2 000 000	ABR	–	–	–	–
	Rouge-noir	39	162.6	376.3	724.1
	`map`	–	167.1	421.5	830.6
	`unordered_map`	–	94.5	69.2	357.4

5.2 Visualisation

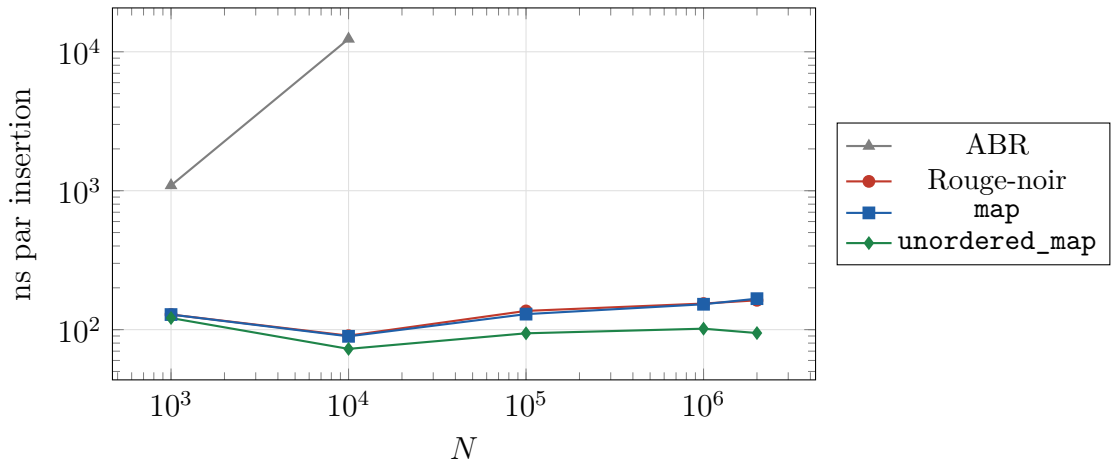


FIGURE 5.1 – Insertion, clés en ordre croissant : l'ABR dégénère, les autres restent plats.

5.3 Analyse

— **L'ABR dégénère** : chaque clé est plus grande que toutes les précédentes et part toujours à droite. L'arbre devient une liste (hauteur $= N$), et quand N est multiplié par 10, le temps par opération est aussi multiplié par environ 12. À $N = 10\,000$, il est **environ 135 fois plus lent** à l'insertion que le rouge-noir (12 377 contre 91 ns), **380 fois** à la recherche (15 724 contre 41 ns) et **60 fois** à la suppression (7 068 contre 115 ns).

— La suppression de l'ABR souffre un peu moins : les clés sont supprimées dans un ordre aléatoire, et la liste raccourcit au fur et à mesure.

— **Le rouge-noir reste équilibré** : 39 niveaux pour deux millions de clés, sous la borne $2\log_2(n+1) \approx 42$. Les rotations corrigent la pente à mesure.

— Notre rouge-noir et `map` sont au coude à coude : l'insertion est identique (163 contre 167 ns à 2×10^6) ; notre rouge-noir cherche un peu plus vite (376 contre 422 ns) et supprime un peu plus vite (724 contre 831 ns).

— L'insertion croissante est **nettement plus rapide** qu'en ordre aléatoire pour les arbres équilibrés (163 contre 742 ns à 2×10^6 pour le rouge-noir) : la descente suit toujours le même chemin, à droite, que le processeur garde en cache et prévoit sans erreur.

Chapitre 6

Résultats : collisions dans la table de hachage

6.1 Données brutes

TABLE 6.1 – Collisions, $N \leq 10^5$: temps moyen par opération (ns), k = nombre de clés par case. En rouge, les cas où `unordered_map` est plus lente que le rouge-noir.

N	Structure	Hauteur	Insertion	Recherche	Suppression
1 000	ABR	22	133.3	30.8	91.3
	Rouge-noir	12	129.2	19.4	82.8
	`map`	–	139.4	22.2	109.4
	`unordered_map` $k = 1$	–	74.6	7.2	42.2
	`unordered_map` $k = 10$	–	115.3	45.0	68.3
	`unordered_map` $k = 100$	–	423.6	362.9	222.1
	`unordered_map` $k = 1000$	–	3 765.5	3 548.0	1 715.8
10 000	ABR	29	152.1	59.5	130.3
	Rouge-noir	16	134.2	40.8	124.8
	`map`	–	143.8	49.6	159.6
	`unordered_map` $k = 1$	–	78.6	9.0	54.6
	`unordered_map` $k = 10$	–	108.5	59.7	82.3
	`unordered_map` $k = 100$	–	447.9	385.5	249.8
	`unordered_map` $k = 1000$	–	3 374.6	3 409.2	1 755.4
100 000	ABR	41	215.6	116.1	220.3
	Rouge-noir	20	185.6	76.9	193.7
	`map`	–	194.4	83.4	229.9
	`unordered_map` $k = 1$	–	74.0	15.9	72.7
	`unordered_map` $k = 10$	–	163.8	98.3	137.4
	`unordered_map` $k = 100$	–	767.5	685.1	392.5
	`unordered_map` $k = 1000$	–	5 637.5	5 576.0	3 038.1

TABLE 6.2 – Collisions, $N \geq 10^6$ (suite du tableau précédent).

N	Structure	Hauteur	Insertion	Recherche	Suppression
1 000 000	ABR	50	552.4	478.4	582.9
	Rouge-noir	24	570.0	327.3	734.1
	`map`	–	601.9	382.3	847.3
	`unordered_map` $k = 1$	–	118.9	58.2	329.3
	`unordered_map` $k = 10$	–	584.1	560.5	538.7
	`unordered_map` $k = 100$	–	3 803.9	4 197.9	2 265.4
	`unordered_map` $k = 1000$	–	34 150.5	37 435.7	18 088.0
2 000 000	ABR	57	769.6	696.8	809.8
	Rouge-noir	25	750.8	396.0	891.5
	`map`	–	767.9	456.4	996.4
	`unordered_map` $k = 1$	–	160.8	66.2	368.7
	`unordered_map` $k = 10$	–	691.9	640.5	621.6
	`unordered_map` $k = 100$	–	4 460.2	4 602.8	2 568.7
	`unordered_map` $k = 1000$	–	39 952.2	43 971.6	22 987.0

Nombre de cases B de la table : 1 031, 10 273, 107 897, 1 056 323 et 2 144 977 pour les cinq valeurs de N. Les arbres sont mesurés une fois par N, avec les clés du niveau $k = 1000$: leurs temps ne dépendent pas de k.

6.2 Visualisation

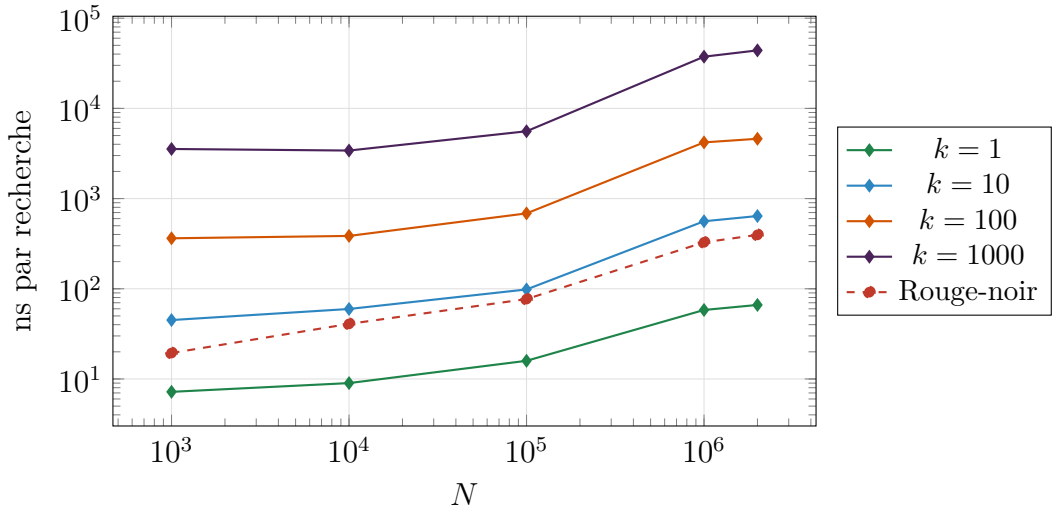


FIGURE 6.1 – Recherche dans `unordered_map` avec k clés par case : chaque niveau de k décale la courbe vers le haut ; le rouge-noir (pointillés) ne bouge pas.

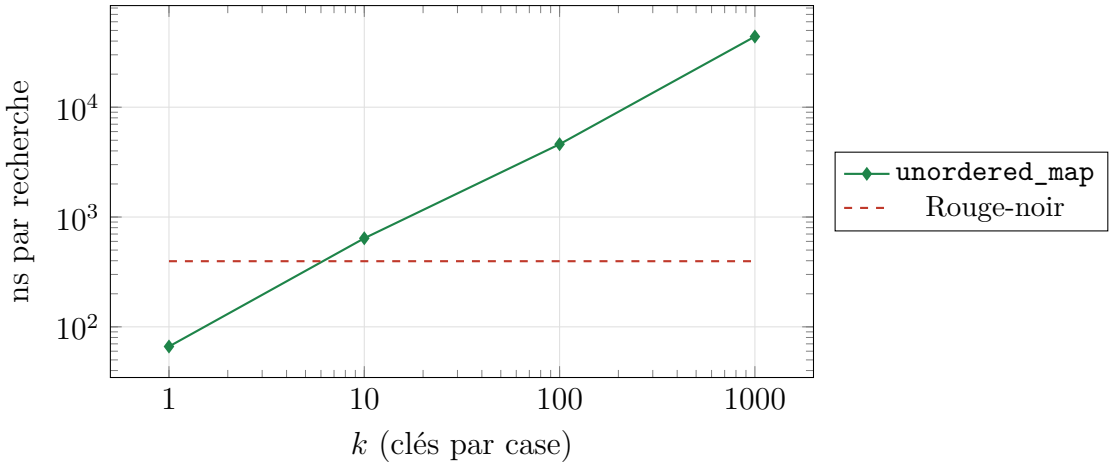


FIGURE 6.2 – Recherche à $N = 2 \times 10^6$ en fonction de k : le temps croît proportionnellement à k (droite de pente 1 en échelle log-log).

6.3 Analyse

— **Le temps croît proportionnellement à k.** À $N = 2 \times 10^6$, la recherche passe de 66 ns ($k = 1$) à 641 ns ($k = 10$), 4 603 ns ($k = 100$) et 43 972 ns ($k = 1000$) : chaque fois que k est multiplié par 10, le temps est multiplié par 7 à 10. C'est le parcours d'une liste de k éléments, soit $k/2$ en moyenne : le coût est bien $O(k)$ par opération, et $O(n)$ quand toutes les clés sont dans la même case.

— **Pour un k donné, le temps dépend peu de N** : à $k = 1000$, la recherche coûte 3 548 ns pour $N = 10^3$ et 5 576 ns pour $N = 10^5$. La hausse à partir de 10^6 (37 436 ns) vient de la mémoire : les nœuds de la liste ne tiennent plus dans le cache du processeur, et chaque maillon visité coûte un accès à la mémoire centrale (environ 80 ns au lieu de quelques nanosecondes).

— **La table de hachage perd son avantage dès $k = 10$** : à la recherche, elle est déjà plus lente que le rouge-noir pour toutes les tailles (641 contre 396 ns à 2×10^6). À $k = 100$, elle est **12 fois plus lente** que lui ; à $k = 1000$, **110 fois plus lente** à la recherche et 53 fois à l'insertion, et **660 fois plus lente qu'elle-même** sans collisions.

— **L'insertion coûte à peu près autant que la recherche** ($k = 1000$: 39 952 contre 43 972 ns) : avant d'ajouter une clé, la table doit parcourir toute la liste pour vérifier qu'elle n'y est pas déjà. **La suppression coûte environ la moitié** (22 987 ns) : les clés sont retirées une à une, donc les listes raccourcissent de k à 0 pendant le test et leur longueur moyenne n'est que de $k/2$.

— **Les arbres ne voient rien.** Les hauteurs (22, 29, 41, 50, 57 pour l'ABR ; 12, 16, 20, 24, 25 pour le rouge-noir) sont exactement celles du scénario aléatoire, et les temps aussi (396 ns pour une recherche dans le rouge-noir à 2×10^6, contre 402 ns en aléatoire). Multiplier les clés par B ne change pas leur ordre relatif, et un arbre ne regarde que cet ordre.

— Ce scénario est artificiel, mais le phénomène est réel : des attaques dites de *hash flooding* envoient à un serveur des clés choisies pour entrer en collision, afin de le ralentir d'un facteur 100 ou 1000. C'est pour cela que certains langages (Python, Rust, Java pour les chaînes) rendent leur fonction de hachage imprévisible.

Chapitre 7

Analyse comparative approfondie

7.1 Notre arbre rouge-noir face à `std::map`

Les deux structures sont des arbres rouge-noir construits avec le même algorithme : elles ont la même forme et la même hauteur, et leurs temps sont **très proches** dans les trois scénarios. Notre version cherche **10 à 15 % plus vite** (402 contre 478 ns à 2×10^6 en aléatoire, 376 contre 422 ns en croissant, 396 contre 456 ns dans le scénario collisions). L'insertion est équivalente (742 contre 801 ns en aléatoire ; 163 contre 167 ns en croissant). Pour la suppression, notre arbre est un peu devant dans les deux cas (928 contre 1 033 ns en aléatoire, 724 contre 831 ns en croissant).

Deux raisons expliquent le petit avantage à la recherche :

— La fonction `map::find` descend toujours jusqu'en bas de l'arbre (recherche de type `lower_bound`), alors que notre recherche **s'arrête dès qu'elle trouve la clé** : en moyenne, un ou deux niveaux de moins.

— Un nœud de `map<int,int>` est plus gros (40 octets, il stocke une paire clé-valeur, contre 32 pour le nôtre) : moins de nœuds tiennent dans le cache.

`std::map` reste plus générale : elle accepte n'importe quel type et n'importe quel comparateur, et fournit des itérateurs. Notre arbre ne stocke que des `int`.

7.2 ABR ou rouge-noir : le prix de l'équilibrage

En ordre aléatoire, l'équilibrage n'apporte qu'un gain modeste : le rouge-noir cherche 1,5 à 1,7 fois plus vite, insère à peu près aussi vite, et **supprime même plus lentement** que l'ABR (13 à 18 %), à cause de la réparation. En ordre croissant, en revanche, il évite un effondrement d'un facteur 60 à 380 dès $N = 10\,000$, et cet effondrement s'aggrave avec N. L'équilibrage est donc une **assurance** : un petit coût dans le cas favorable, contre une protection totale dans le pire cas.

7.3 Les arbres face à la table de hachage

Sans collisions, `unordered_map` est la plus rapide presque partout : à 2×10^6 en aléatoire, elle cherche environ **6 fois plus vite** que le rouge-noir (68 contre 402 ns), insère et supprime 2 à 2,5 fois plus vite. Elle calcule directement la case de la clé au lieu de descendre une vingtaine de niveaux. Seule exception : avec 1 000 clés en aléatoire,

son insertion (129 ns) n'est pas meilleure que celle des arbres, car le coût de création et d'agrandissement de la table pèse plus lourd que le gain du hachage.

Mais cette vitesse repose sur deux conditions. D'abord, elle ne garde **aucun ordre** : impossible d'obtenir les clés triées, le minimum, le maximum ou toutes les clés d'un intervalle. Ensuite, son $O(1)$ **n'est qu'une moyenne**, qui suppose des clés bien réparties : le scénario collisions montre qu'avec 10 clés par case elle est déjà dépassée par le rouge-noir, et qu'avec 1000 clés par case elle est 110 fois plus lente que lui. Le rouge-noir, lui, garantit $O(\log n)$ **quelles que soient les clés**.

7.4 Le pire cas de chaque structure

Table 7.1 – Recherche : cas normal et pire cas de chaque structure (ns, $N = 10^4$ pour l'ABR, 2×10^6 pour les autres).

Structure	Pire cas	Cas normal	Pire cas	Facteur
ABR	clés triées ($N = 10^4$)	58	15 724	$\times$ 270
`unordered_map`	1000 clés par case	68	43 972	$\times$ 640
Rouge-noir	aucun	402	396	$\times$ 1,0

Chaque structure rapide a un talon d'Achille : l'ordre d'arrivée pour l'ABR, la fonction de hachage pour la table. Le rouge-noir est la seule des trois à n'avoir aucun pire cas : ses temps sont les mêmes dans les trois scénarios.

7.5 Mémoire

Table 7.2 – Taille d'un nœud sur une machine 64 bits (hors surcoût de `new`).

Structure	Contenu d'un nœud	Taille
ABR	clé + 2 pointeurs	24 octets
Rouge-noir	clé + couleur + 3 pointeurs	32 octets
`map<int,int>`	couleur + 3 pointeurs + paire clé-valeur	40 octets
`unordered_map<int,int>`	pointeur suivant + paire	16 octets, plus les cases

L'équilibrage coûte un pointeur `parent` et une couleur par nœud, soit 16 Mo de plus pour deux millions de clés : un prix faible face au gain de hauteur.

7.6 Bilan

TABLE 7.3 – Quelle structure choisir ?

Besoin	Structure	Raison
Savoir si une clé existe, clés bien réparties	`unordered_map` ou `unordered_set`	$O(1)$ en moyenne, la plus rapide
Clés triées, minimum, intervalles	`map` / `set`	arbre rouge-noir, $O(\log n)$ garanti
Temps garanti, clés fournies par un tiers	`map` / `set`	insensible aux collisions
Données pouvant arriver triées	jamais un ABR simple	dégénère en liste
Comprendre l'équilibrage	arbre écrit à la main	tout le fonctionnement est visible

Chapitre 8

Conclusion

8.1 Synthèse des résultats

1. Notre arbre rouge-noir est correct : il donne les mêmes arbres que les exercices faits à la main, et ses règles sont vérifiées automatiquement après les insertions et après les suppressions.
2. Avec des clés triées, l'ABR devient une liste : à $N = 10\,000$, il est environ **135 fois plus lent** à l'insertion, **380 fois** à la recherche et **60 fois** à la suppression que le rouge-noir, qui garde une hauteur de 39 pour deux millions de clés.
3. Avec des clés mélangées, le rouge-noir **cherche 1,5 à 1,7 fois plus vite** que l'ABR, insère à peu près aussi vite, mais supprime environ 15 % plus lentement, à cause de la réparation des couleurs.
4. Notre rouge-noir et `std::map` sont équivalents ; le nôtre cherche 10 à 15 % plus vite, car il s'arrête dès qu'il trouve la clé, et supprime un peu plus vite.
5. Sans collisions, `unordered_map` est la plus rapide (environ 6 fois plus rapide que le rouge-noir pour chercher), mais ne garde aucun ordre.
6. Avec des collisions, son temps croît proportionnellement au nombre de clés par case : dès 10 clés par case, elle est dépassée par le rouge-noir ; à 1000 clés par case, elle est **110 fois plus lente** que lui. Le rouge-noir, lui, ne change pas.

8.2 Recommandations

— Pour un dictionnaire sans besoin d'ordre, avec des clés bien réparties : la table `unordered_map`.

— Pour des données triées, des requêtes par intervalle, ou un temps garanti face à des clés qu'on ne contrôle pas : `map`, qui garantit $O(\log n)$ quel que soit l'ordre d'arrivée et quelles que soient les clés.

— Un ABR simple ne convient que si l'on sait que les données arrivent dans un ordre aléatoire.

— Pour des mesures plus stables, répéter le benchmark davantage de fois et fermer les autres programmes : les mesures sous Windows sont un peu bruitées.

8.3 Conclusion finale

L'équilibrage ne rend pas chaque opération plus rapide : en ordre aléatoire, l'ABR insère aussi vite que le rouge-noir et supprime même plus vite, et la table de hachage bat tout le monde. Son vrai rôle est de donner une **garantie** : quel que soit l'ordre d'arrivée des données, et quelles que soient leurs valeurs, la hauteur reste en $O(\log n)$. Cette garantie évite l'effondrement de l'ABR face à des données triées et celui de la table de hachage face aux collisions, pour un coût de quelques recolorations, d'au plus deux rotations par insertion et trois par suppression, et d'un pointeur de plus par nœud. Chaque structure a son pire cas ; le rouge-noir est la seule à ne pas en avoir.